Ideassion Technology Solutions

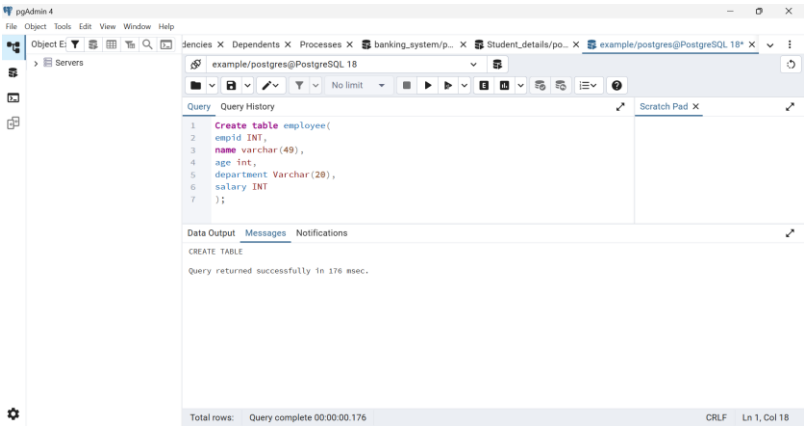
Name: Durga R

Domain: Springboot

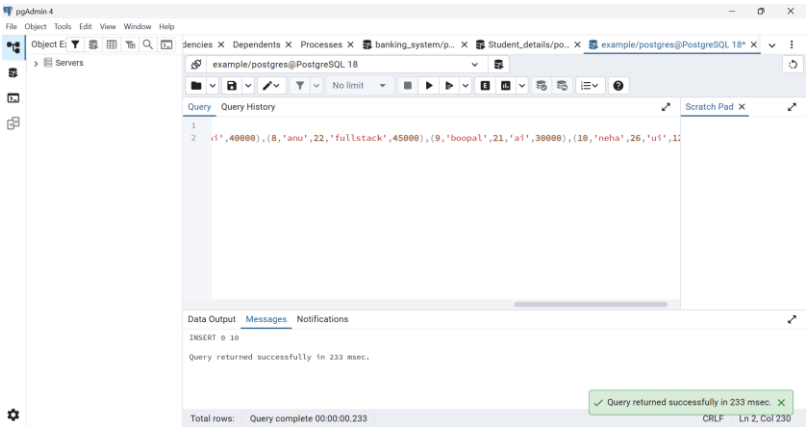
SQL Coding Practice

1. Employee Table (CRUD Operations)*

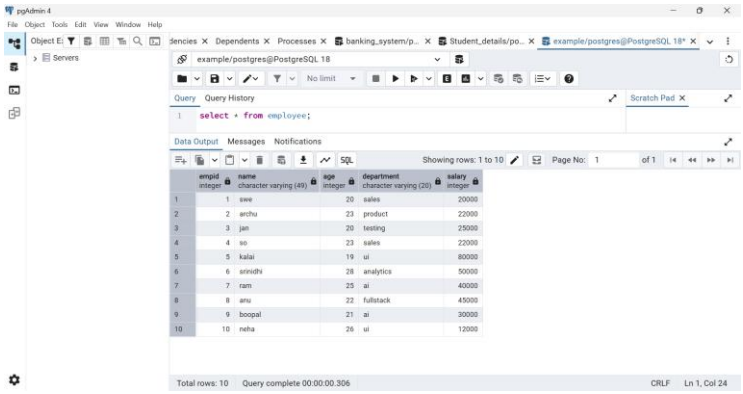
1.Create an *Employee* table.



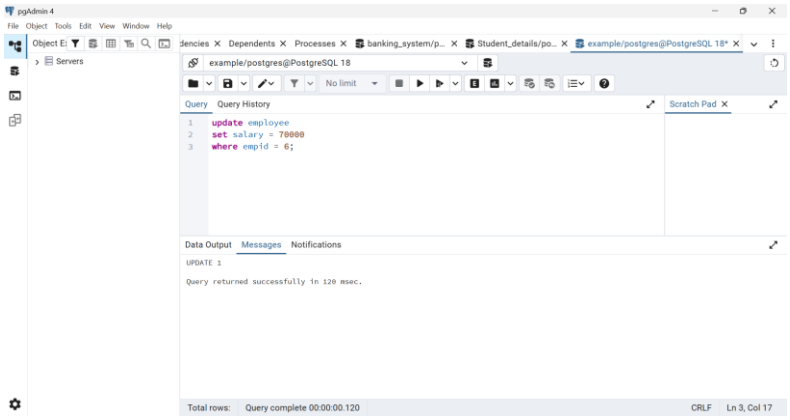
2. Insert at least *10 employee records*.

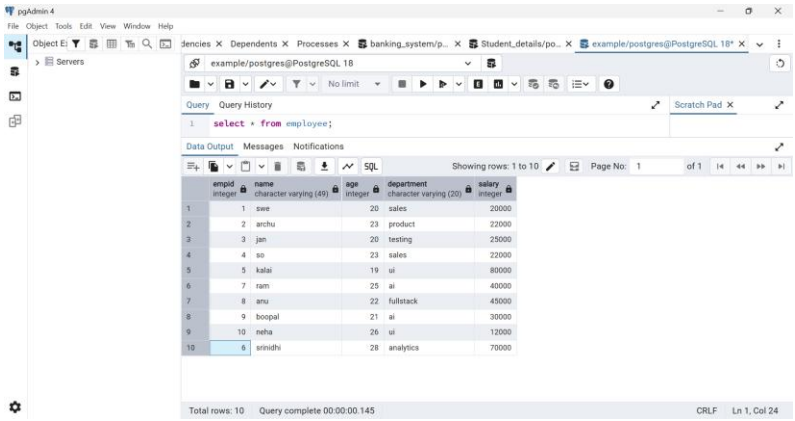


3.Display all records.

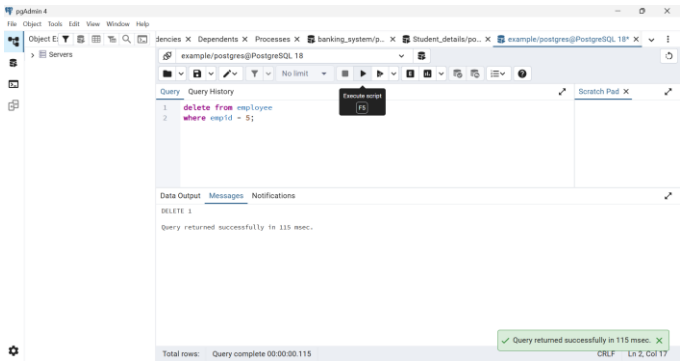


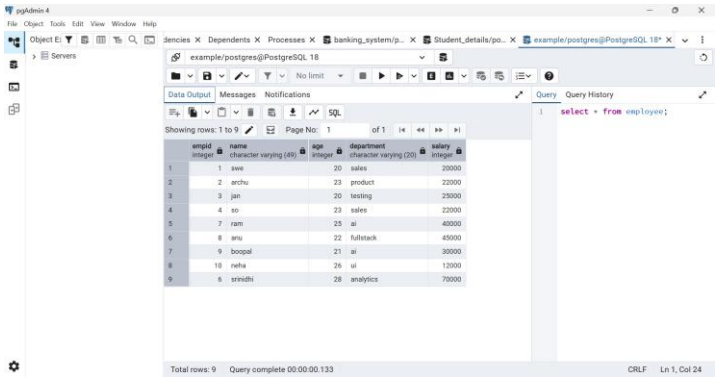
4.Update an employee's salary.



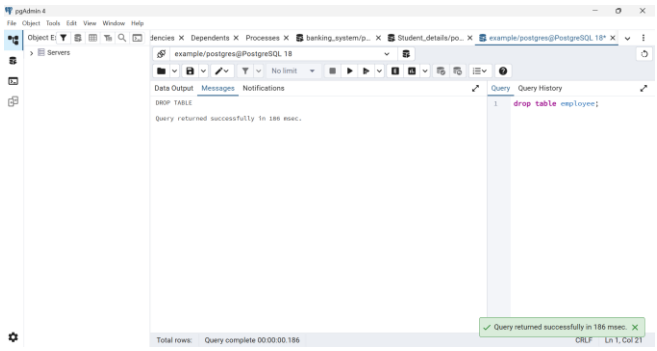


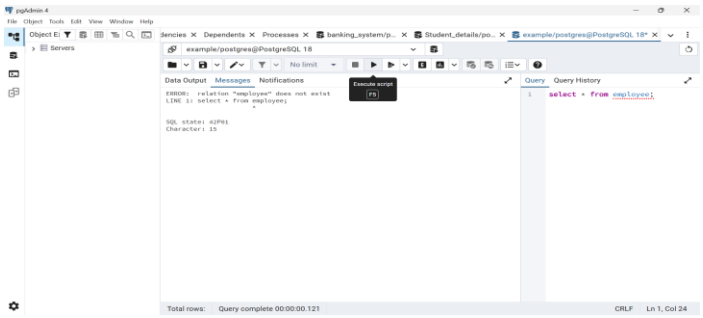
5.Delete one employee record.





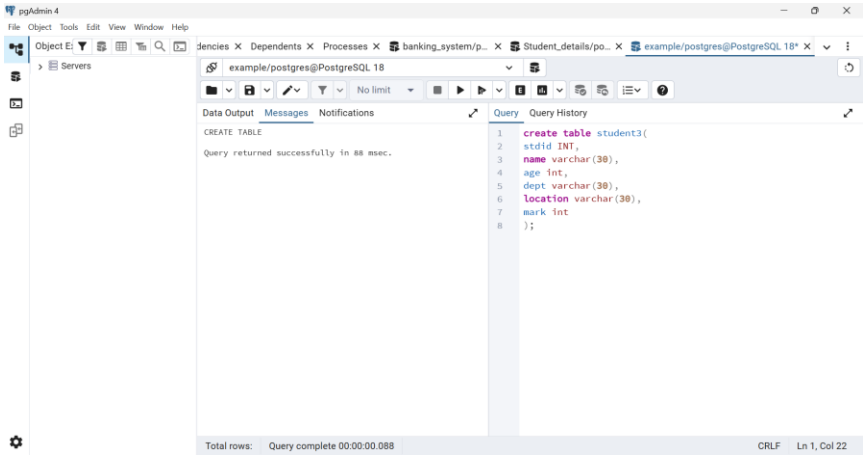
6.Drop the table.



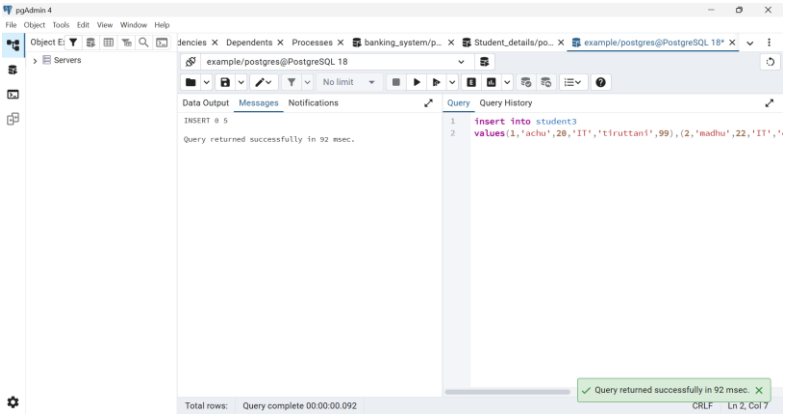


2. Student Table*

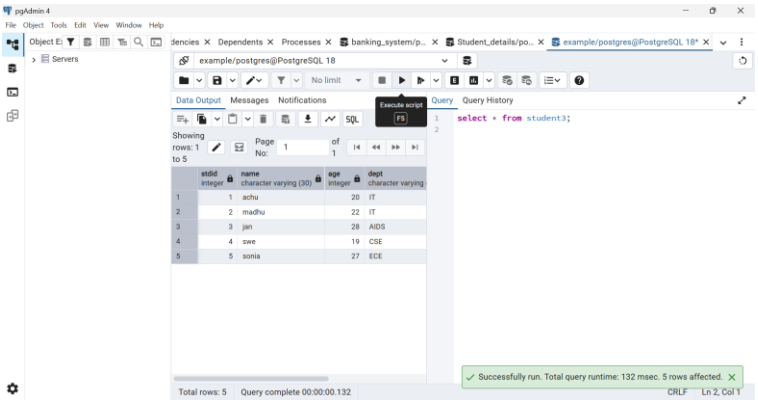
1. Create a *Student* table.



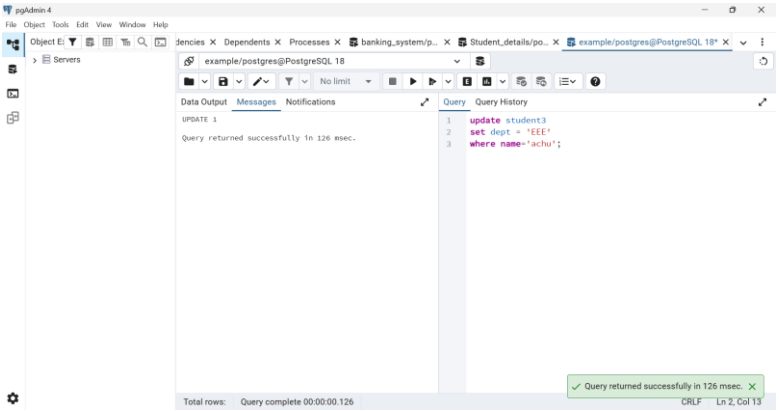
2.Insert at least *10 student records*.

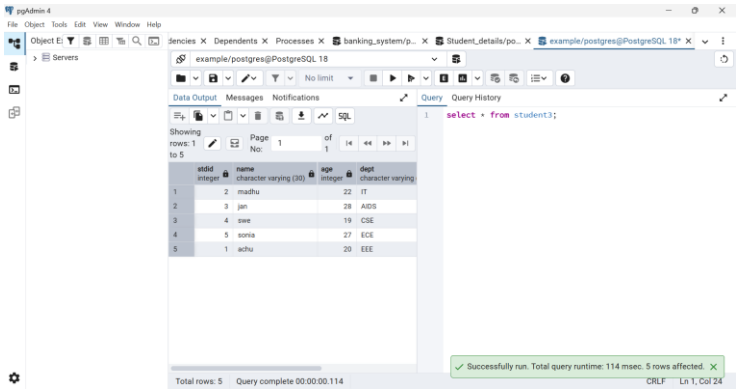


3.Display all students.

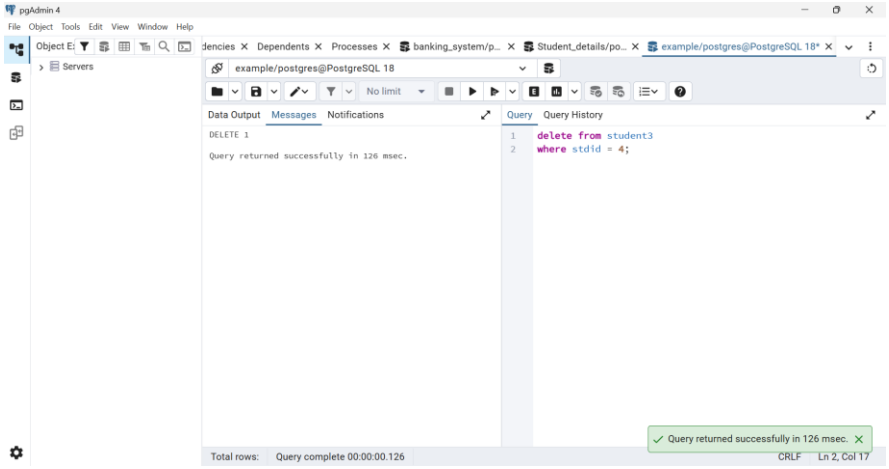


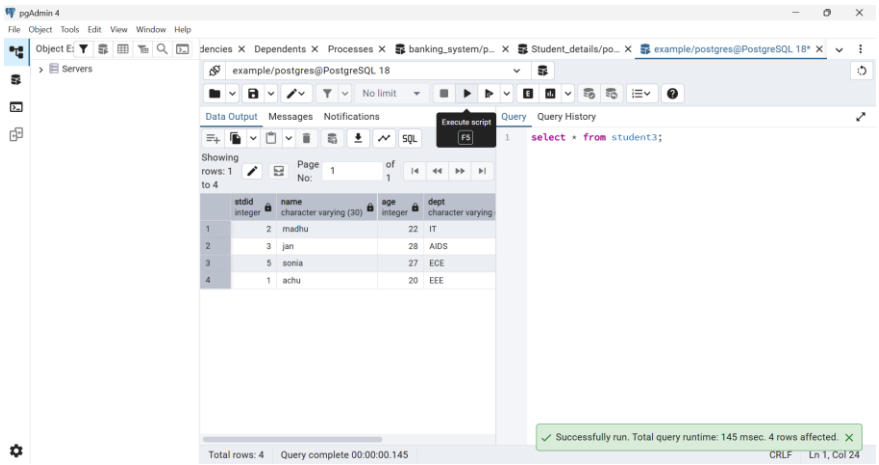
4.Update one student's department.



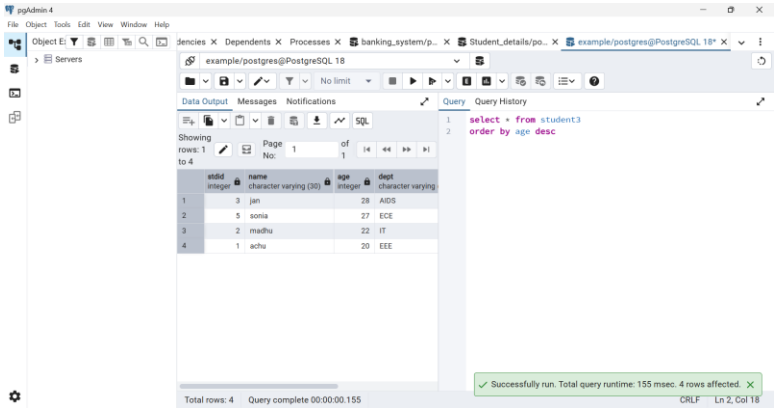


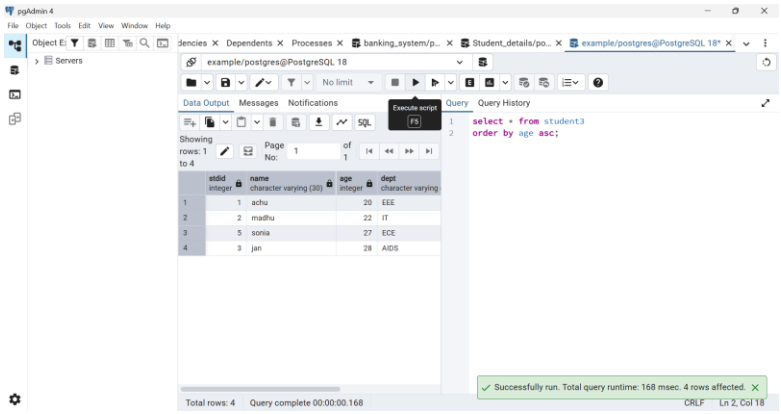
5.Delete one student record.



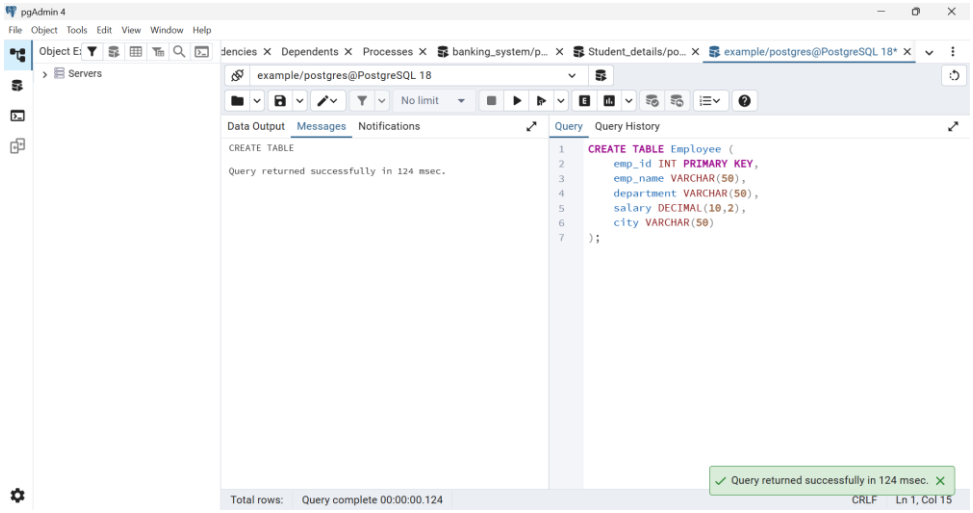


6.Sort students by name.





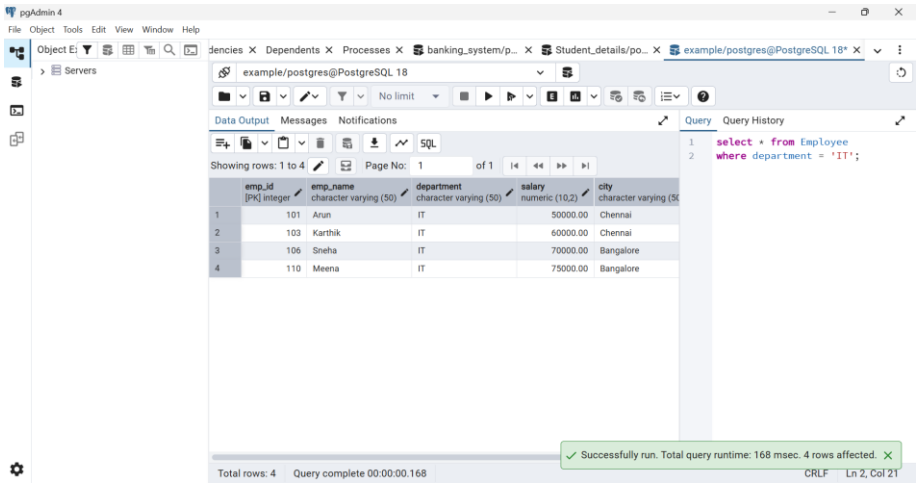
3. SQL Filtering



WHERE:

select * from Employee

where department = 'IT';



LIKE:

SELECT * FROM Employee

WHERE emp_name LIKE 'Ra%';

And (‘A__’)

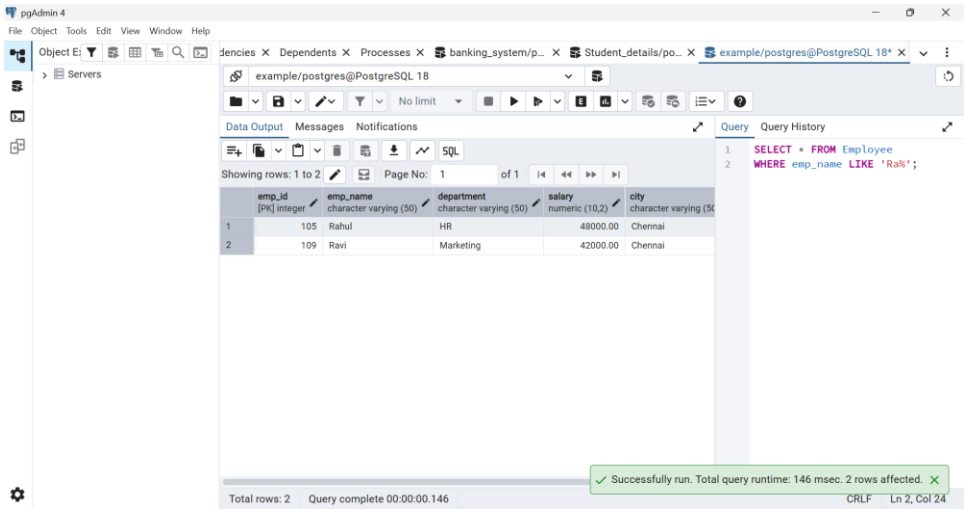
SELECT * FROM Employee

WHERE emp_name LIKE 'R___';

And

SELECT * FROM Employee

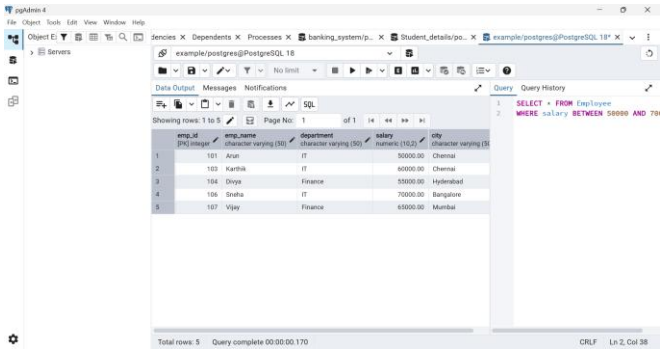
WHERE emp_name LIKE '_a__';



BETWEEN:

SELECT * FROM Employee

WHERE salary BETWEEN 50000 AND 70000;

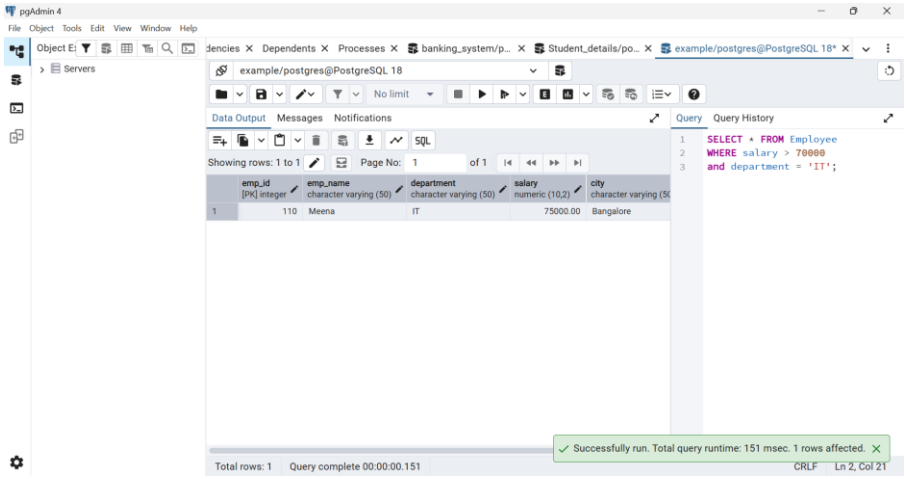


AND:

SELECT * FROM Employee

WHERE salary > 70000

and department = 'IT';



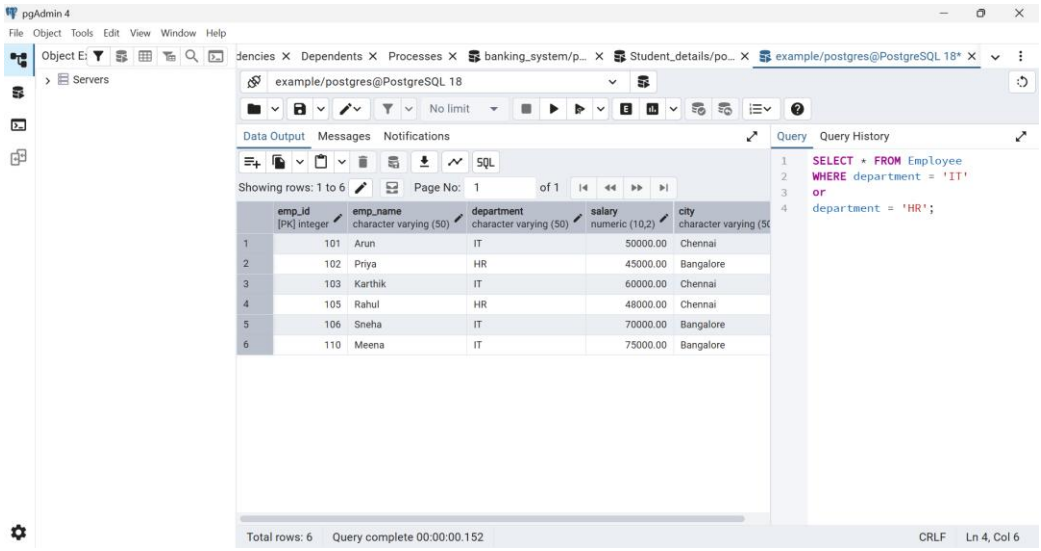
OR:

SELECT * FROM Employee

WHERE department = 'IT'

or

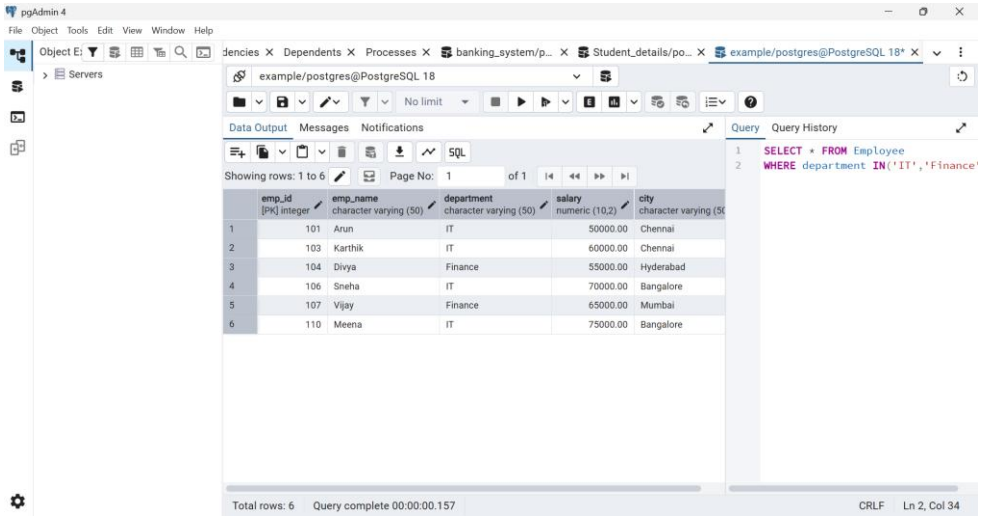
department = 'HR';



IN:

SELECT * FROM Employee

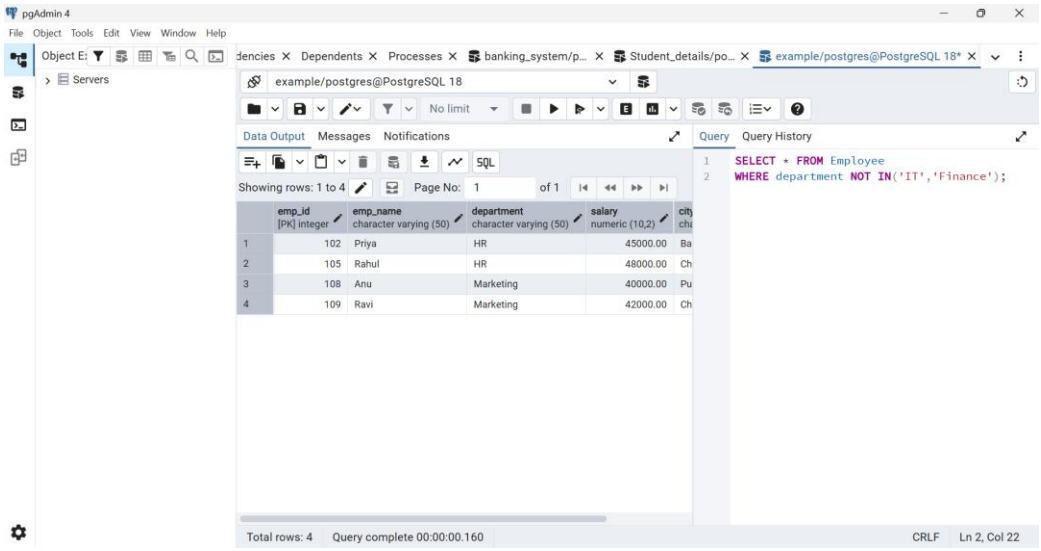
WHERE department IN('IT','Finance');



NOT IN:

SELECT * FROM Employee

WHERE department NOT IN('IT','Finance');

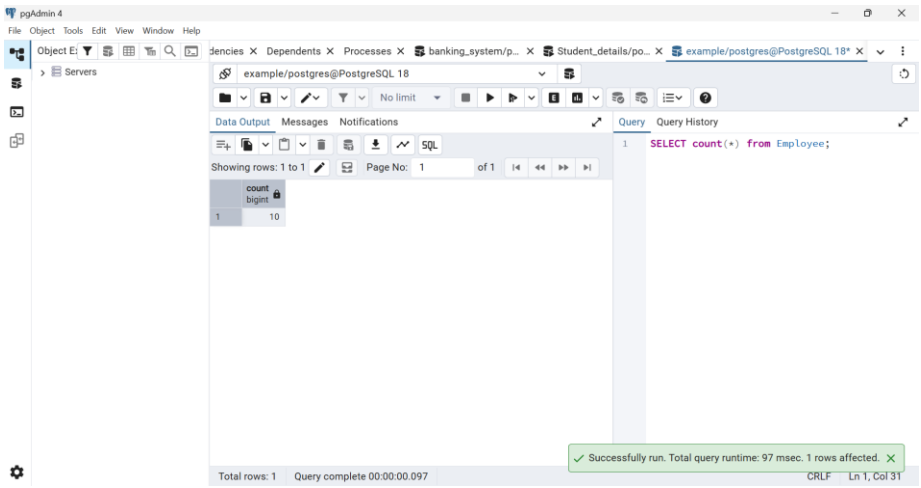


4. Aggregate Functions

Write queries using:

1. COUNT()

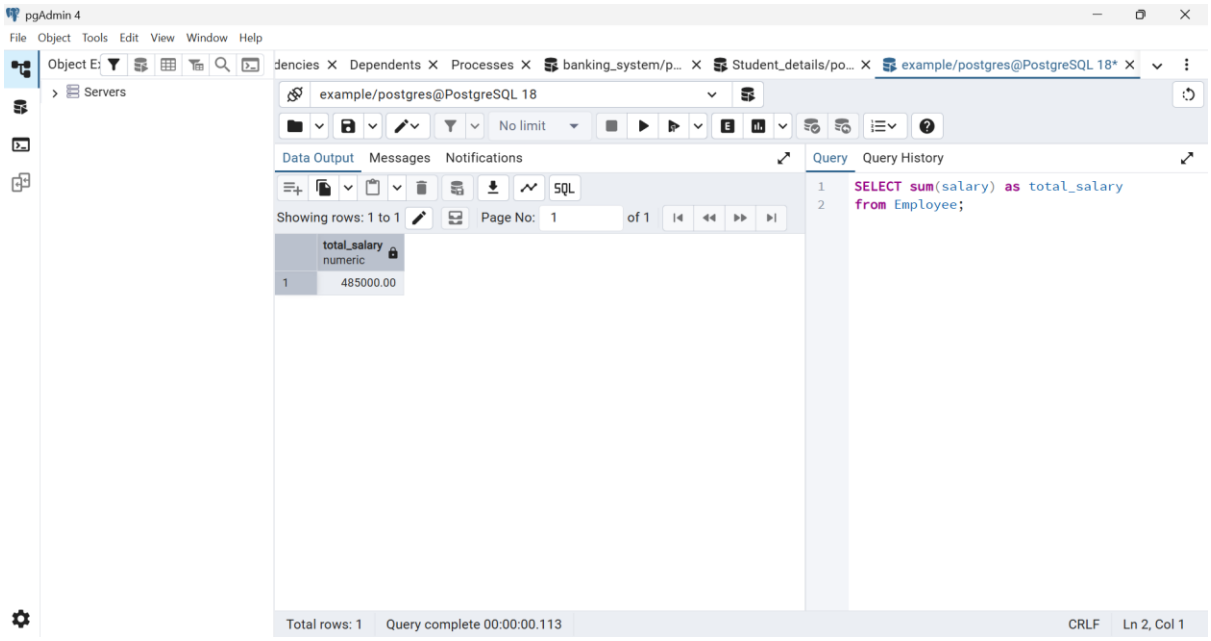
SELECT count(*) from Employee;



2.SUM()

SELECT sum(salary) as total_salary

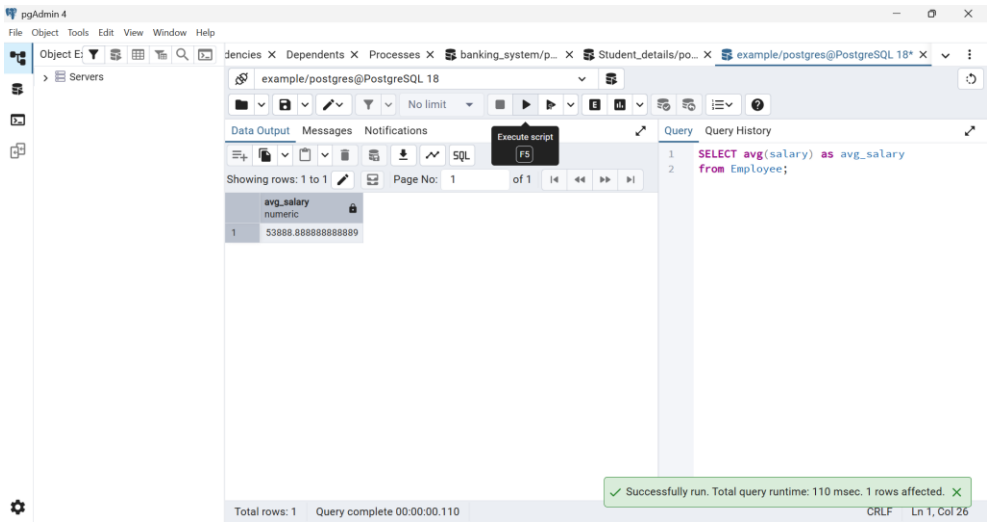
from Employee;



3.AVG()

SELECT avg(salary) as avg_salary

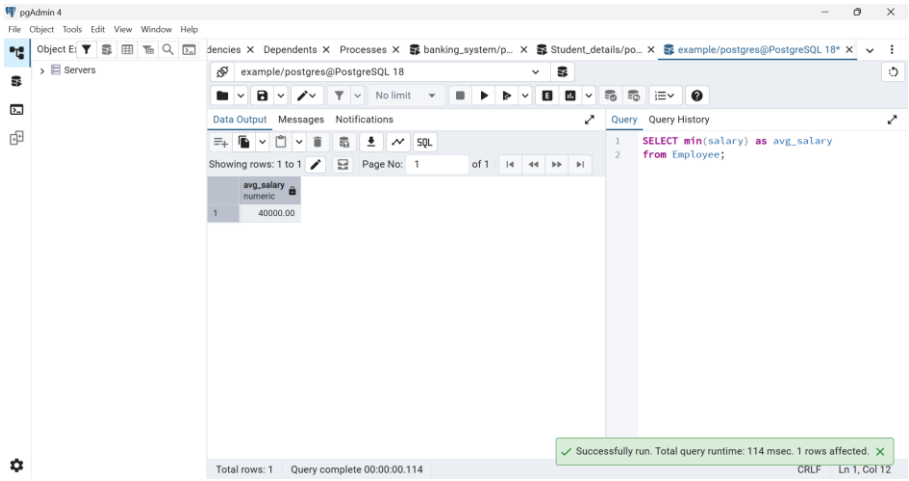
from Employee;



4.MIN()

SELECT min(salary) as avg_salary

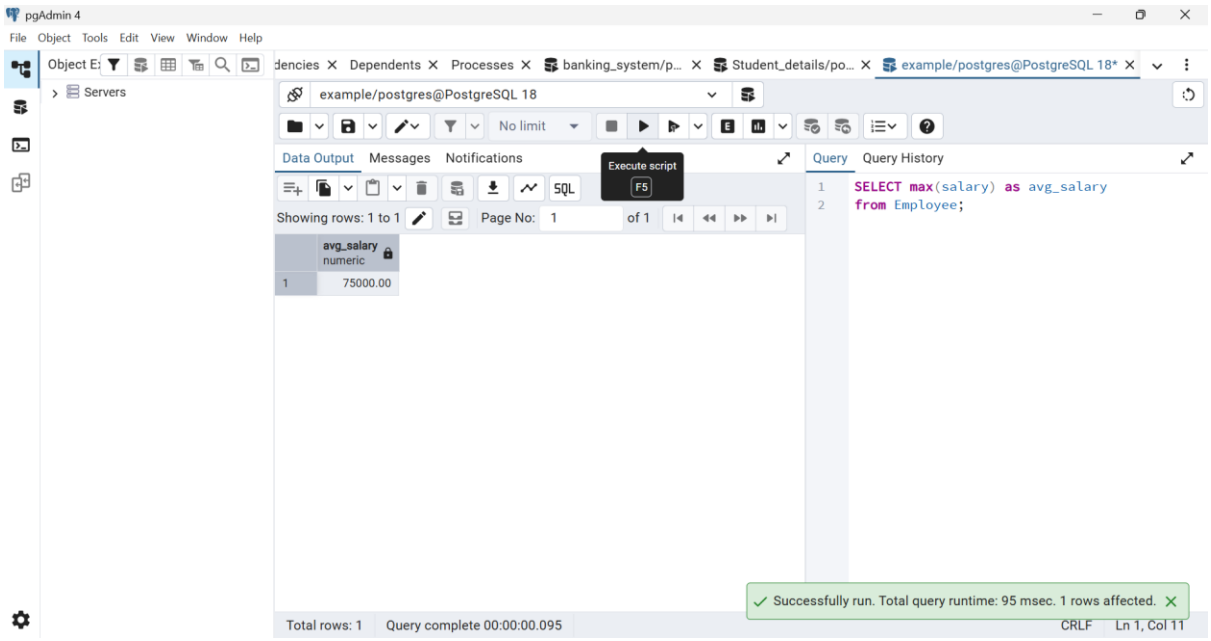
from Employee;



5.MAX()

SELECT max(salary) as avg_salary

from Employee;



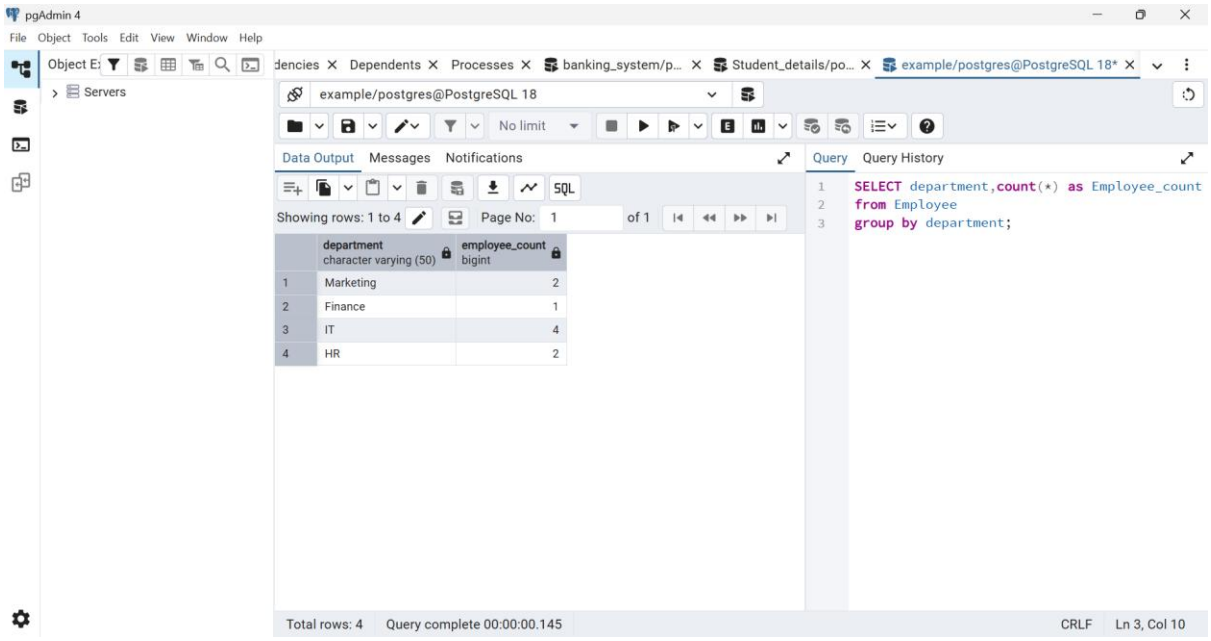
5. GROUP BY & HAVING*

1) **Count employees in each department.**

SELECT department,count(*) as Employee_count

from Employee

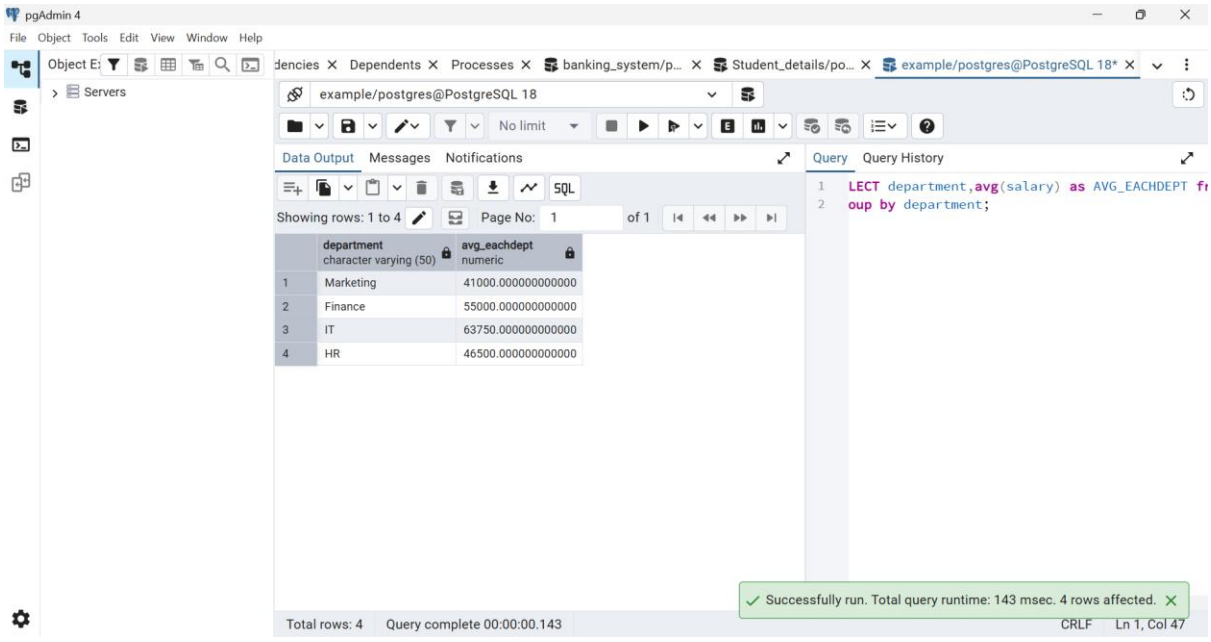
group by department;



2) Find the average salary for each department.

SELECT department,avg(salary) as AVG_EACHDEPT from Employee

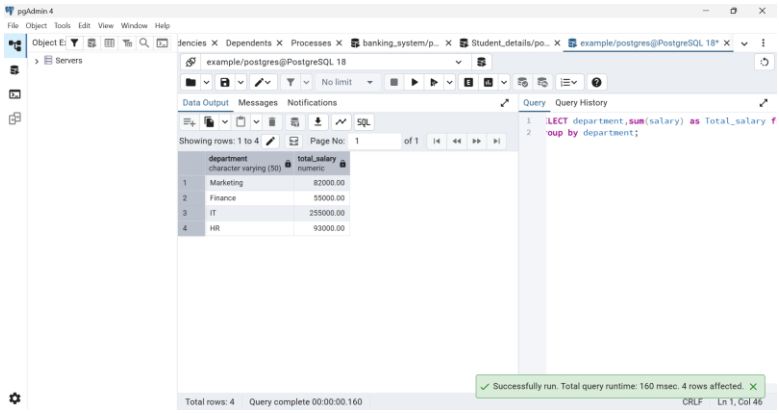
group by department;



3) Find the total salary for each department.

SELECT department,sum(salary) as Total_salary from Employee

group by department;



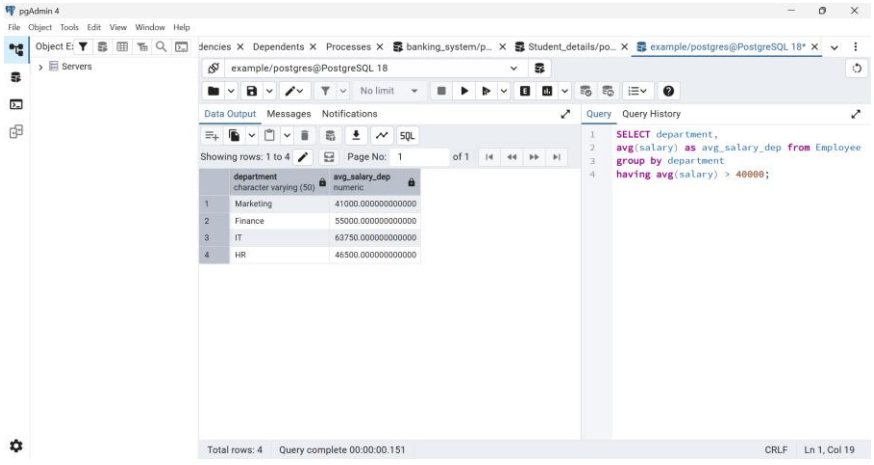
4) Display only departments whose average salary is greater than a specific value.

SELECT department,

avg(salary) as avg_salary_dep from Employee

group by department

having avg(salary) > 40000;



6. SQL JOINs

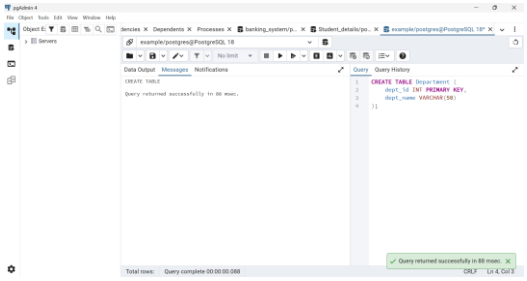
Create *Employee* and *Department* tables and practice:

*** INNER JOIN**

SELECT e.emp_id,

e.emp_name,

```
    e.department,
    d.location
FROM Employee e
INNER JOIN Department d
ON e.department = d.dept_name;
```

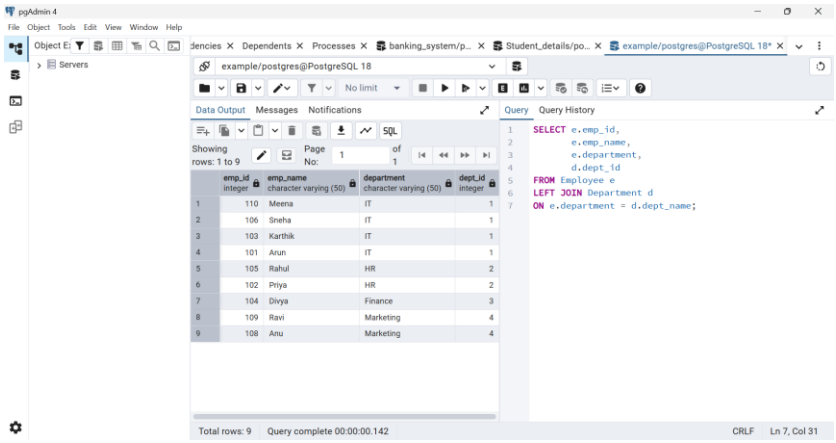


Returns only matching records from both tables.

* LEFT JOIN

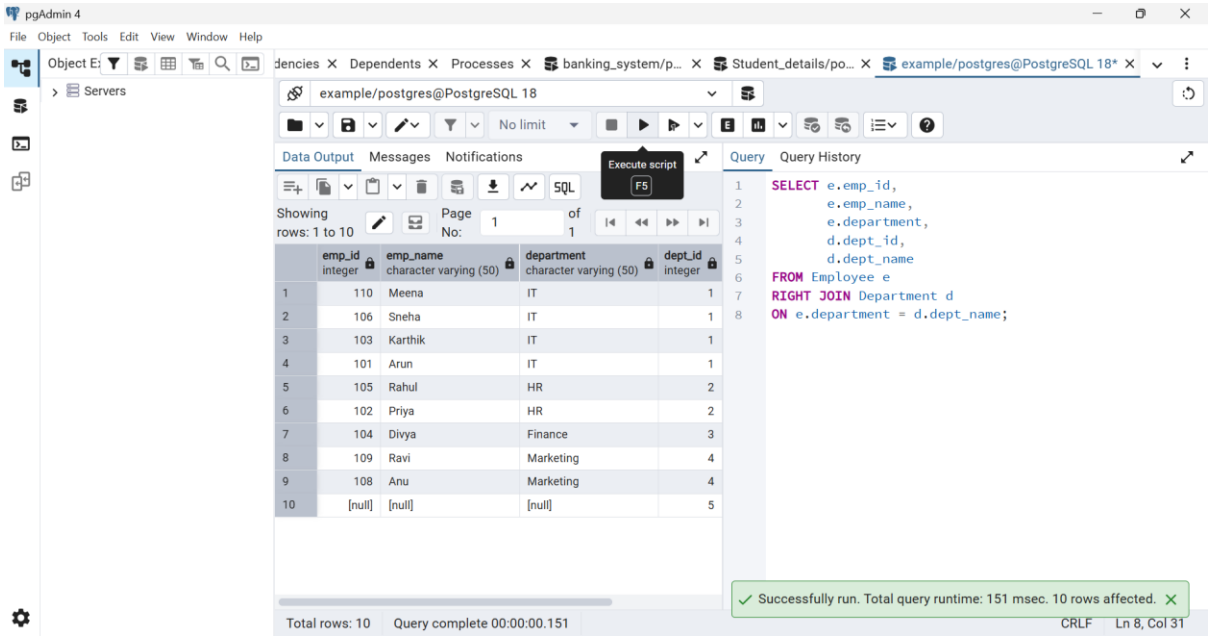
Returns all employees and matching department details.

```
SELECT e.emp_id,
    e.emp_name,
    e.department,
    d.location
FROM Employee e
LEFT JOIN Department d
ON e.department = d.dept_name;
```



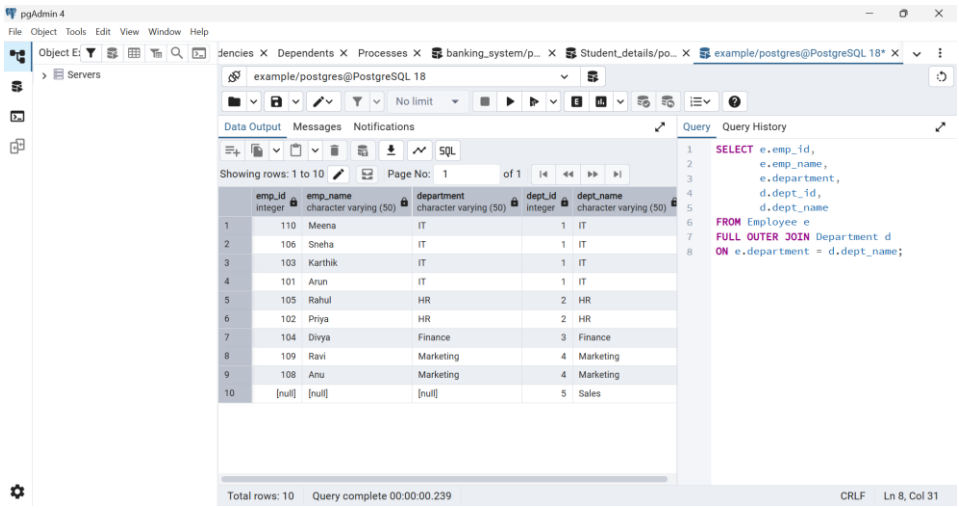
* RIGHT JOIN

```
SELECT e.emp_id,
       e.emp_name,
       e.department,
       d.dept_id,
       d.dept_name
FROM Employee e
RIGHT JOIN Department d
ON e.department = d.dept_name;
```



*** FULL OUTER JOIN**

```
SELECT e.emp_id,
    e.emp_name,
    e.department,
    d.dept_id,
    d.dept_name
FROM Employee e
FULL OUTER JOIN Department d
ON e.department = d.dept_name;
```



7. Customer & Orders*

Create *Customer* and *Orders* tables and write queries to:

CREATE TABLE Customer (customer_id INT PRIMARY KEY, customer_name VARCHAR(50), city VARCHAR(50));

INSERT INTO Customer VALUES

(1,'Arun','Chennai'),

(2,'Priya','Bangalore'),

(3,'Rahul','Hyderabad'),

(4,'Divya','Mumbai'),

(5,'Sneha','Pune');

*** Display customer orders.**

```
SELECT c.customer_id,
    c.customer_name,
    o.order_id,
    o.amount
FROM Customer c
INNER JOIN Orders o
ON c.customer_id = o.customer_id;
```

*** Find customers without orders.**

```
SELECT c.customer_id,
    c.customer_name
FROM Customer c
LEFT JOIN Orders o
ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

*** Calculate the total purchase amount for each customer.**

```
SELECT c.customer_name,
    SUM(o.amount) AS total_purchase
FROM Customer c
```

```
INNER JOIN Orders o
ON c.customer_id = o.customer_id
GROUP BY c.customer_name;
```

*** Find the customer who spent the most.**

```
SELECT c.customer_name,
       SUM(o.amount) AS total_spent
FROM Customer c
INNER JOIN Orders o
ON c.customer_id = o.customer_id
GROUP BY c.customer_name
ORDER BY total_spent DESC
LIMIT 1;
```

8. Advanced SQL Queries*

Write queries to:

*** Find the second highest salary.**

```
SELECT MAX(salary) AS second_highest_salary
FROM Employee
WHERE salary < (
   SELECT MAX(salary)
   FROM Employee
);
```

*** Find the third highest salary.**

```
SELECT MAX(salary) AS third_highest_salary
```

```
FROM Employee
WHERE salary < (
   SELECT MAX(salary)
   FROM Employee
   WHERE salary < (
      SELECT MAX(salary)
      FROM Employee
   )
);
```

*** Display the top 5 highest-paid employees.**

```
SELECT *
FROM Employee
ORDER BY salary DESC
LIMIT 5;
```

* Find employees earning more than the average salary.

9. Subqueries*

Practice writing subqueries to:

*** Find employees earning above the average salary.**

```
SELECT emp_name,
      salary
FROM Employee
WHERE salary > (
   SELECT AVG(salary)
```

```
    FROM Employee
);
```

*** Find the department with the highest average salary.**

```
SELECT department,
       AVG(salary) AS avg_salary
FROM Employee
GROUP BY department
ORDER BY avg_salary DESC
LIMIT 1;
```

*** Compare employee salaries within departments.**

```
SELECT emp_name,
       department,
       salary
FROM Employee e
WHERE salary >
(
    SELECT AVG(salary)
    FROM Employee
    WHERE department = e.department
);
```

10. Mini SQL Challenge

Solve the following:

*** Find the department with the highest average salary.**

```
SELECT department,
    AVG(salary) AS avg_salary
FROM Employee
GROUP BY department
ORDER BY avg_salary DESC
LIMIT 1;
```

*** Find employees earning more than their department's average salary.**

```
SELECT emp_name,
    department,
    salary
FROM Employee e
WHERE salary >
(
  SELECT AVG(salary)
  FROM Employee
  WHERE department = e.department
);
```

*** Find customers who placed more than two orders.**

```
SELECT emp_name,
    department,
    salary
FROM Employee e
WHERE salary >
(
```

```
    SELECT AVG(salary)
    FROM Employee
    WHERE department = e.department
);
```

*** Find products that have never been ordered.**

```
SELECT *
FROM Product
WHERE product_id NOT IN
(
    SELECT product_id
    FROM Order_Items
);
```

*** Find the second highest salary *without using LIMIT***.

```
SELECT MAX(salary) AS second_highest_salary
FROM Employee
WHERE salary <
(
    SELECT MAX(salary)
    FROM Employee
);
```